# L'OPTION V1 — Phase 0 Audit and Implementation Plan

Branch: `claude/peaceful-shannon-ewahm3` · Base commit: `0cb0fe0` (single "Initial commit") Scope: read-only inspection. No application code, schema, or deployed resource was changed in Phase 0.

## 0. What was and was not verifiable

| Item | Status |
|---|---|
| Repository source (app/, src/, config) | Fully inspected |
| `npm ci` from lockfile | **Ran — succeeded** (662 packages) |
| `tsc --noEmit` | **Ran — 1 error** (`src/services/notificationService.ts:8`, `NotificationBehavior` missing `shouldShowBanner`/`shouldShowList`) |
| `expo export --platform android` (JS bundle) | **Ran — succeeded** (with dummy `EXPO_PUBLIC_*` values) |
| Lint | **Not available** — no ESLint config or script in the repo |
| Automated tests | **None exist** — no test runner, no test files |
| Native/EAS build | **Not run** — expected to fail at prebuild: `app.json:50` references `./assets/notification.wav`, which does not exist |
| Deployed Supabase schema, RLS, triggers, functions, storage policies | **BLOCKED** — no Supabase URL/keys/DB connection in this environment |
| Payment provider | **BLOCKED** — no provider chosen; no payment code exists in the app |

**The repository contains no SQL, no migrations, no `supabase/` directory, no Edge Functions, and no `.env.example`.** Every table, column, policy, and trigger the app relies on exists (if at all) only in the deployed project. Everything in §3 about the database is inferred from client queries and must be verified against the live project before Phase 1 migrations are written.

## 1. Architecture and dependency map

**Stack (locked versions):** Expo SDK 57.0.22, React Native 0.86.3 (New Architecture on, JSC engine), React 19.2.3, expo-router 57 (typed routes on), `@supabase/supabase-js` **2.39.8** (old, Mar-2024 line), zustand 5, TypeScript 6.0.3, expo-location, expo-notifications, react-native-maps.

**Shape:** a single Expo app with role-segmented route groups; all business logic runs in the client and talks to Supabase directly with the anon key + user JWT. There is no trusted backend layer.

```
app/_layout.tsx  ── useProtectedRoute() → routes by authStore.role
 ├─ (auth)/          login, signup                  → authStore
 ├─ onboarding       nutrition profile              → user_profiles
 ├─ (app)/           customer: menu, cart, checkout, orders, order-tracking, profile,
 │                   upgrade / my-plan / progress / reorder (subscription & mock features)
 ├─ (collaborator)/  partner: orders, orders/[id], earnings, profile → collaboratorStore
 ├─ (driver)/        dashboard, orders/[id], history, earnings, profile
 └─ (admin)/         overview, orders, drivers, collaborators, meals, earnings

src/lib/supabase.ts         anon client, AsyncStorage session
src/stores/                 authStore, cartStore (persisted), userStore, adminStore (mostly unused),
                            collaboratorStore, driverEarningsStore (unused), subscriptionStore
```

```
src/services/                locationService (unused duplicate), notificationService (client-side Expo
push)
```

**Tables/buckets referenced by the client** (existence and shape unverified): `user_profiles`, `meals`, `orders`, `order_items`, `collaborators`, `collaborator_orders`, `collaborator_payouts`, `drivers`, `driver_weekly_earnings`, `cash_collections`, `user_push_tokens`, `driver_push_tokens`, `admin_logs`, `subscriptions`; storage bucket `meal-images`. Realtime on `orders` and `collaborator_orders`.

**Implied order flow today:** checkout inserts `orders` (status `confirmed`) → separate insert of `order_items` → client fires push to all online drivers → driver claims (`driver_id` null → self) → driver sets `picked_up` / `on_the_way` / `delivered` → optional `cash_collections` row. `collaborator_orders` is **never created by the app**; partner assignment either happens via an unknown DB trigger or not at all.

## 2. Verified defects

Severity: **C** critical (security/financial integrity), **H** high (core flow broken or inconsistent), **M** medium.

### 2.1 Auth, roles, and access control

| # | Sev | Location | Defect |
|---|---|---|---|
| A1 | C | `src/stores/authStore.ts:38-60` | `signUp(email, password, role, …)` inserts `user_profiles` with a **client-chosen `role` and `is_admin: role === 'admin'`**. Any client can call this with `'admin'`. Safety depends entirely on unverified RLS. |
| A2 | C | `src/stores/userStore.ts:95-103`, `app/onboarding.tsx:113-127` | Customer updates `user_profiles` with an arbitrary partial object on the same row that holds `role`/`is_admin`. Unless RLS/column grants block it, a user can promote themselves via `update({ role: 'admin', is_admin: true })`. |
| A3 | C | `src/stores/adminStore.ts:90-123`, `src/components/admin/AdminAuthModal.tsx` | The "security phrase" is **never checked** — `verifyAdminAuth(securityPhrase)` ignores its argument and only re-reads `is_admin`. The modal is security theatre and logs a false `admin_login`. |
| A4 | C | All `app/(admin)/*` screens | Admin authority is enforced only by client routing. Admin screens call `supabase.from(...).update/delete` directly (e.g. `app/(admin)/drivers.tsx:72-75`, `orders.tsx:76-80`, `meals.tsx:243`). Real protection depends on unverified RLS. Most admin actions write **no audit log** (only the unused `adminStore` logs, from the client, which is forgeable). |
| A5 | H | `src/stores/authStore.ts:105-172` | Role resolution is a client-side cascade (`user_profiles.role` → `collaborators` → `drivers` → default `customer`); errors silently fall back to `customer`. Role is not in JWT claims, so RLS cannot rely on it consistently. |
| A6 | H | `src/stores/authStore.ts:74-92` | `onAuthStateChange` is registered inside `loadSession` with no unsubscribe; the `SIGNED_OUT` event does not clear role; every `TOKEN_REFRESHED` re-runs role queries. Logout (`:69-72`) does not clear `userStore.profile`, the persisted cart, collaborator/admin stores, or push tokens → data leaks between accounts on a shared device. |
| A7 | M | `src/lib/supabase.ts` | Missing the documented React Native setup: `detectSessionInUrl: false`, `react-native-url-polyfill/auto` import (dependency installed, never imported), and `AppState`-driven `startAutoRefresh/stopAutoRefresh`. Non-null asserted env vars with no validation → opaque crash if unset. SDK 2.39.8 should be upgraded to current 2.x and retested. |
| A8 | M | `app/_layout.tsx:12-128` | Route guard uses an `isChecking` ref that can drop a re-check; `app/index.tsx` always redirects to `/(app)` before role is known (briefly mounts customer screens for every role). |

| # | Sev | Location | Defect |
|---|---|---|---|
| A9 | H | `app/(admin)/collaborators.tsx:84-88` | Partner provisioning looks up `user_profiles.user_id = <email>` — always fails (uuid vs email). Partner onboarding via the UI is broken. `:55-58` embeds `users:user_id (email)`, which PostgREST cannot resolve against `auth.users`. |
| A10 | H | `app/(driver)/index.tsx:228-235`, `app/(admin)/drivers.tsx:41-75` | `drivers.status` mixes **approval** (`pending`/`rejected`) with **availability** (`online`/`offline`). A pending or rejected driver can set their own status to `online`, which is the same state as "approved". There is no driver sign-up flow in the app at all. |

## 2.2 Checkout, pricing, and orders

| # | Sev | Location | Defect |
|---|---|---|---|
| O1 | C | `app/(app)/checkout.tsx:30-31,53-66` | Prices, subtotal, delivery fee (hard-coded 15), and total are computed from the **persisted client cart** and written straight into `orders`. A client can submit any price. No service fee (8 MAD) is applied. |
| O2 | C | `app/(app)/checkout.tsx:56-60` | Client sets `status: 'confirmed'` and `payment_method` (including `'card'`) itself. Selecting "Credit/Debit Card" creates a confirmed order with **no payment** taken. |
| O3 | H | `app/(app)/checkout.tsx:85-95` | Order and items are two separate inserts; an items failure leaves an orphan confirmed order visible to drivers. Item `price` comes from the client cart, not `meals`. No idempotency key → double-tap creates duplicate orders. Order number is `Date.now()+random` (collision-possible, not DB-generated). |
| O4 | H | `app/(app)/checkout.tsx:98-104`, `src/services/notificationService.ts:124-141` | The customer client enumerates `drivers` and sends Expo pushes to them. This requires customers to read driver rows and push tokens. |
| O5 | H | — | No customer cancellation, no partner-rejection fallback, no delivery-failure path anywhere in the app. Driver history filters `cancelled`/`rejected` statuses nothing ever sets. |
| O6 | H | `app/(driver)/orders/[id].tsx:82,366-370` | Driver screen shows a "Service Fee" defaulting to **19 MAD** and "Total to Collect" = `total + 19`, an amount the customer was never quoted. |

## 2.3 Partner preparation

| # | Sev | Location | Defect |
|---|---|---|---|
| P1 | H | `app/(app)/checkout.tsx` (absence) | Nothing creates `collaborator_orders`; partner assignment is undefined in code. |
| P2 | H | `src/stores/collaboratorStore.ts:202-217` | Partner accept/prepare/ready only updates `collaborator_orders.preparation_status`; `orders.status` never changes. Customer tracking and drivers never see `preparing`/`ready`. Drivers can claim orders before they are accepted or ready. |
| P3 | M | `src/stores/collaboratorStore.ts:228` vs `app/(collaborator)/orders/[id].tsx:192` | Rejection reason written to `rejection_reason`, displayed from `preparation_notes` (always blank). No state guard — any status can be written from any status. |
| P4 | M | `src/stores/collaboratorStore.ts:127-128` | `acceptanceRate: 98.5` and `avgPrepTime: 24` are hard-coded mock values shown as real stats. |

## 2.4 Driver lifecycle and location

| # | Sev | Location | Defect |
|---|---|---|---|
| D1 | H | `app/(driver)/index.tsx:252-282` | Claim uses a conditional update (`driver_id IS NULL`) — good against double-claim — but does not check the driver is approved, online, has no active job, or that the order is ready/assigned to a partner. Relies on an RLS policy that must allow a driver to update *unassigned* orders, which typically also lets them edit other columns (price, totals) on any unassigned order. |
| D2 | H | `app/(driver)/orders/[id].tsx:119-141` | Driver writes `orders.status` to any value with no transition check and no `driver_id` filter in the query. |
| D3 | C | `app/(driver)/orders/[id].tsx:143-185` | Cash collection records `amount = delivery_fee` (15) instead of the amount collected from the customer, then auto-marks delivered. The cash ledger cannot reconcile to what was collected. Driver "commission" is 15% of the delivery fee here but the dashboard (`index.tsx:183-210`) reports the full delivery fee as earnings — three different definitions of driver pay. |
| D4 | M | `app/(driver)/index.tsx:52` | Realtime filter `driver_id=is.null` is not a supported Realtime filter operator; the "new order" listener does not work as intended. Available orders are not refreshed on status changes. |
| D5 | M | `app/(driver)/index.tsx:100-125` vs `src/services/locationService.ts` | Two location implementations; the dashboard one captures a stale `activeOrder` closure, so the order location update depends on render timing. Tracking stops on unmount but the DB status remains `online`. Location is written into the `orders` row every 5 s, which re-broadcasts the whole order row on Realtime. No retention/cleanup. |
| D6 | M | `app/(driver)/orders/[id].tsx:44-47`, `app/(collaborator)/orders/[id].tsx:46-49` | `subscribeToOrderUpdates()` returns a cleanup that `useEffect` discards → channels leak and duplicate on each visit. |
| D7 | M | `app/(app)/order-tracking/[id].tsx:29-34,111` | Customer timeline lacks `accepted`/`picked_up`; for those states `findIndex` returns −1 and the timeline shows nothing completed. Location subscription re-created on every order update (effect depends on `order` object). No reconnect/refetch logic anywhere. |

## 2.5 Notifications, financial records, mock features, hygiene

| # | Sev | Location | Defect |
|---|---|---|---|
| N1 | H | `src/services/notificationService.ts:83-167` | Push sending happens from the client; any client that can read tokens can push arbitrary messages to any user. `upsert` (`:70`) has no `onConflict`; `.single()` lookups (`:93,114`) fail for multiple devices. Login stores token only when not `__DEV__`. TS error at `:8`. |
| F1 | C | — | No payment provider, webhook, payment state, or immutable ledger exists. "Revenue" on admin screens (`app/(admin)/index.tsx:59-75`, `earnings.tsx:38-50`) sums `orders.total` of **all** orders including undelivered/unpaid. `collaborator_share`/`loption_share` origin is unknown (no code computes them). |
| X1 | M | `app/(app)/upgrade.tsx:41`, `app/webview/index.tsx`, `app/_layout.tsx:147-173`, `subscriptionStore.ts`, `app/(app)/profile.tsx:29` (`streakDays = 7`), `app/(app)/reorder.tsx:24-26` (hard-coded order), `website/index.html:197` | Out-of-scope subscription/streak/reorder features are live in the V1 UI, some with fake data. The deep link `loption://subscription/active` shows "Subscription Activated" to anyone who opens it. |

| # | Sev | Location | Defect |
|---|---|---|---|
| X2 | M | `app/(driver)/extractor.py`, `app/(driver)/output.txt` (2,465 lines), `project_tree_*.txt`, `assets/icon_BAK.png`, `assets/ChatGPT Image …png` | Dev artefacts committed inside the route tree; `output.txt` is a stale copy of driver screens. |
| X3 | M | `app.json:50` | Missing `notification.wav` → native build will fail. |

**Secrets:** scanned the tree and git history (single commit) for JWTs, `service_role`, `sk_` keys — **none found**. `.gitignore` excludes `.env*`. No `.env.example` documents required variables. The anon key is (correctly) the only key the client expects.

## 3. Unknowns requiring access

**Supabase (need read access to the project, or a `supabase db dump --schema-only` + policy export):** 1. Actual columns, types, FKs, constraints and indexes of every table in §1 (e.g. is `drivers.id` = `auth.users.id`? does `orders.platform_service_fee` exist? `order_items.price` numeric or int?). 2. Whether RLS is enabled per table and the exact policies — this decides whether A1, A2, A4, D1, D2, O1 are *exploitable now* or only latent. 3. Triggers/functions: is there a trigger that creates `collaborator_orders`, computes shares, sends notifications, or creates `user_profiles` on sign-up? 4. Realtime publication membership and whether Realtime RLS is honoured for `orders`. 5. Storage bucket `meal-images` policies (who can upload/overwrite — `upsert: true` is used). 6. Auth settings: email confirmation on/off, how admin/driver/partner accounts were created historically. 7. Data volume and dirty data (duplicate drivers rows — `index.tsx:139-153` hints at this; invalid statuses) — needed to make migrations safe.

**Payment provider:** provider identity (CMI, Payzone, YouCan Pay, other), sandbox credentials, webhook signature scheme, refund API, fees, merchant account structure, and whether split settlement to two accounts is contractually supported.

## 4. Business decisions needed from the CTO

1. **Delivery fee:** confirm 15 MAD flat (or zone/distance-based). **Service fee** 8 MAD confirmed? Any tax (TVA) shown separately? Any minimum order?
2. **Payment methods at V1 launch:** cash-on-delivery only until the gateway is approved (recommended), or block launch on card?
3. **Partner assignment rule:** single partner for all orders, nearest branch, customer-chosen branch, or admin-assigned? Can an order contain meals from more than one partner? (Recommend: one partner per order.)
4. **Partner economics:** what is `revenue_share_percent` a percentage *of* — meal subtotal only? Is it partner share or platform commission?
5. **Driver pay:** flat per delivery, full delivery fee, or 15% of delivery fee? (Code currently uses all three.) Who holds collected cash, and how is it remitted?
6. **Driver rules:** max active jobs (recommend 1), may a driver claim before the meal is `ready`, is the claim model open-pool or admin/auto-dispatch?
7. **Cancellation:** until which state may a customer cancel (recommend: before partner accepts)? Refund policy for card orders cancelled later? Who bears cost of a delivery failure (customer absent)?
8. **Account provisioning:** partners and drivers created only by admin invite (recommend), or self-registration with approval?
9. **Location data:** retention period for driver location history (recommend: keep only latest position + per-order trail purged after N days).
10. **Mock features:** confirm hide-not-delete for subscription/upgrade/my-plan/progress/streak/reorder screens in V1.

11. **Access:** grant a staging Supabase project (and read-only access to production schema) so the deployed state can be diffed before migrations.

## 5. Proposed migrations (Phase 1–3), pending schema verification

Placed under `supabase/migrations/` with timestamped names, applied with the Supabase CLI. Step 0 is a **baseline**: dump the deployed schema into `0000_baseline.sql` so later migrations are diffs against reality, not guesses.

| Migration | Purpose |
|---|---|
| `baseline` | Captured deployed schema (no-op on existing env; creates it on a clean DB). |
| `roles_and_profiles` | `app_role` enum; `user_roles` table writable only by admin/service role; `handle_new_user` trigger creating a `customer` profile; revoke `UPDATE (role, is_admin)` on `user_profiles` from `authenticated`; `is_admin()`, `current_role()` SECURITY DEFINER helpers; optional custom-access-token hook to put role in JWT. |
| `partners_drivers` | Split `drivers.approval_status` (pending/approved/suspended/rejected) from `availability` (online/offline); FKs to `auth.users`; uniqueness on `drivers.id` / `collaborators.user_id`; dedupe step for existing duplicates. |
| `rls_policies` | Enable RLS and write explicit per-role policies on every table; no direct client `INSERT/UPDATE` on `orders`, `order_items`, `collaborator_orders`, `cash_collections`, ledger tables. |
| `order_state_machine` | `order_status` enum, `order_status_history`, `order_events` audit table; `transition_order(order_id, to_status, reason)` function enforcing allowed transitions per role; CHECK constraints and indexes (`orders(status)`, `orders(driver_id, status)`, `collaborator_orders(collaborator_id, status)`). |
| `create_order_rpc` | `create_order(items jsonb, address, payment_method, idempotency_key)` — reads prices from `meals`, snapshots name/price into `order_items`, computes fees from `pricing_config`, assigns partner, all in one transaction; unique `(user_id, idempotency_key)`. |
| `driver_claim_rpc` | `claim_order(order_id)` with `SELECT … FOR UPDATE SKIP LOCKED`, approval/online/active-job checks; `driver_locations` table (latest position) separate from `orders`, with retention job. |
| `pricing_config` | Versioned server-side fee configuration (delivery fee, service fee) referenced by each order. |
| `payments_ledger` | `payments`, `payment_events` (unique provider event id for replay protection), append-only `ledger_entries` (no UPDATE/DELETE grants), `payouts`, reconciliation views. |
| `notifications_outbox` | `push_tokens` (unique per device), `notification_outbox` processed by an Edge Function; failures never roll back order state. |

## 6. Implementation sequence

1. **Phase 1 (security first):** obtain schema dump → baseline migration → roles/profile lockdown → RLS for every table → fix `supabase.ts` RN setup, auth listener lifecycle, logout clearing all stores → fix partner/driver provisioning → admin actions via SECURITY DEFINER RPCs with audit log → remove fake admin phrase modal → add `.env.example` → add a test harness: local Postgres (Docker/`supabase start`, both available in this container) running pgTAP or SQL-based authorization tests for each role (permitted + forbidden), plus Jest for client logic → add ESLint + `typecheck` script.
2. **Phase 2:** state machine + `create_order`/`transition_order`/`claim_order`/`cancel_order` RPCs; rewire checkout, partner, driver, customer, admin screens to RPCs; concurrency tests (parallel claims).
3. **Phase 3:** `pricing_config` (8 MAD service fee; delivery fee per decision); payment provider interface + sandbox adapter; Edge Functions for checkout session and webhook with signature verification and idempotency; ledger; cash-collection as a server-recorded event with the correct amount. **No live payments without CTO approval.**

4. **Phase 4:** partner assignment, driver eligibility, `driver_locations`, resilient Realtime (refetch on (re)subscribe/AppState foreground), server-side push via outbox.
5. **Phase 5:** hide V1-out-of-scope screens, remove dev artefacts, unify earnings terminology, deployment/rollback/monitoring docs, staging E2E run.

## 7. Recommended next task

Before writing any Phase 1 migration: provide staging Supabase access (or a schema-only dump including policies, triggers and functions from the deployed project) and answer decisions 1–8. With that in hand, Phase 1 starts with the baseline migration and the role/RLS lockdown (A1–A4), which are the highest-risk findings.